Unit Test Automation Summary Report: CTFd

Prepared by: M. Taha Course: SQE, Assignment 3 (Build Unit Test Automation Solution) Institution: FAST-NUCES Date: 18 October 2026 Group repository: https://github.com/taharehann/ctfd-automation-testing

This report covers my part of the group assignment: the shared setup work (Task 1) and the three modules assigned to me (Authentication, Challenges and Scoreboard). My group partner's three modules are documented separately in the same repository.

1. Overview

The goal of this assignment was to pick an open-source Python project and build an automated test solution for it. I chose CTFd, a Capture-The-Flag platform written in Python with Flask, and tested it through its web interface using Selenium WebDriver and Python's unittest framework.

In total, my part contains 49 automated tests across three modules plus one smoke test. All 49 passed in the final full run, which used three parallel workers on Windows. The test design notes, the test data files, an Excel test case sheet, setup videos and run instructions are all committed to the group repository.

2. Project selection

The assignment asked for a Python project with at least six modules that is easy to set up. I picked CTFd (version 3.8.8) for these reasons:

- Setup is short. It needs a clone, a pip install and one command to start, and it stores data in SQLite, so no database server is needed.
- It has a full browser interface with forms, buttons, tables and an admin panel, which suits Selenium.
- Its features are separate enough to be split fairly between two people: registration and login, challenges, the scoreboard, user profiles, teams and administration.

I took Authentication, Challenges and Scoreboard. My partner took the other three areas.

3. Environment and setup (Task 1)

The test environment was set up on a Windows machine with Python 3.12, Google Chrome and a virtual environment for each of the two projects. CTFd runs from its own folder and the test project sits next to it.

Setup steps:

1. Clone CTFd, create a virtual environment, install its requirements and start it with `python serve.py`. It then runs at http://localhost:4000.
2. Complete the one-time setup page: event name TestCTF, Users mode, an admin account, and no email verification.
3. Log in as admin and create one challenge named Test Challenge (category Web, 100 points, flag `flag{test123}`, case sensitive, visible, unlimited attempts).
4. In the test repository, install the test dependencies (`selenium` and `ddt`) from `requirements.txt`.

Two screen recordings of this process are in the `docs` folder: `Setup_video.mp4` and `Setup_video_Part2.mp4`. Full written instructions are in the repository README.

4. Modules and test design (Task 2)

For each module I wrote design notes before writing any tests. The notes list the rules found in the application, the techniques used, and the inputs and expected results. They are in the `notes` folder.

Module	Techniques used	Automated tests
Authentication: registration	Equivalence class partitioning, boundary value analysis, decision table, error guessing	21
Authentication: login and logout	Equivalence class partitioning, state transition	6
Challenges: flag submission	Equivalence class partitioning, error guessing, state transition	12
Scoreboard	Equivalence class partitioning, boundary value analysis, use case testing	9

Module	Techniques used	Automated tests
Setup check	Smoke test	1
Total		49

Authentication. I read the validation code in CTFd's `auth.py` and confirmed each rule in the browser. Usernames and passwords are trimmed of spaces, passwords can be at most 128 characters, usernames cannot look like an email address, and emails are lowercased before the duplicate check. From this I built equivalence classes for each field and boundary cases at 0, 1, 2, 127, 128 and 129 characters. Login tests cover a username or an email with the right password, a wrong password, an unknown user and empty fields, plus a logout test.

Challenges. The flag box was tested with the correct flag, a wrong flag, the right text in the wrong case, a partial flag, extra characters, spaces around the flag, an empty flag, a SQL-style string and a 1000-character string. Three state transition tests check how the page behaves before and after a challenge is solved.

Scoreboard. These tests check that the page opens, that the table has the Place, User and Score columns, that a user with no solves is not listed (0 points) while a user who solved the 100 point challenge is listed, that scores are in descending order, that places are numbered without gaps, and that a username link opens the user's profile.

5. How the tests are built

- **Tools:** Selenium WebDriver with Chrome, Python unittest, and the `ddt` library for data-driven tests.
- **Independence:** every challenge and scoreboard test registers its own new user with a random name. Once a user solves a challenge, the messages on the page change, so a shared user would make results depend on earlier runs. Fresh users keep each test self-contained.
- **Waiting:** tests wait for the page to show what they need, such as a message or a table header, instead of relying on fixed pauses.
- **Watching a run:** a `STEP_DELAY` setting adds short pauses so a viewer can follow the inputs and results. It can be set to 0 for fast runs.

6. Test data and Excel sheet (Task 3)

The registration, login and flag tests read their inputs from CSV files in the `data` folder: `register_data.csv` (21 rows), `login_data.csv` (5 rows) and `flag_data.csv` (9 rows). Each row becomes its own test. Values that must be unique, such as usernames, use placeholders that are replaced with random values at run time.

The Excel file `docs/CTFd_Test_Cases.xlsx` lists all 49 test cases with their module, design reference, technique, description, input, expected result, data source, automated test name and status. It also contains copies of the three CSV files and a summary sheet whose totals are calculated by formulas.

7. Configuration and execution (Task 4)

The configuration is in `test_config.py` and is used by the runner `run_tests.py`.

- **Groups:** smoke, auth, challenges and scoreboard.
- **Suites:** `quick` (smoke and auth) and `full` (all four groups).
- **Order:** groups run in the order they appear in the configuration file, and the tests inside a class follow an ordered list in the same file.
- **Parameterization:** the CSV-driven tests described above.
- **Parallel execution:** with `--workers N`, the tests of a group run at the same time, each in its own browser.

Common commands:

- `python run_tests.py --list` shows all groups and tests in run order without running them.
- `python run_tests.py --suite quick` runs the quick suite.
- `python run_tests.py --group challenges` runs one group.
- `python run_tests.py --workers 3 --delay 0` runs everything in parallel.
- `python run_tests.py --delay 2` runs everything slowly, for demonstration.

8. Results

The final full run used `python run_tests.py --workers 3 --delay 0`.

Group	Tests	Passed	Failed
Smoke	1	1	0
Authentication	27	27	0
Challenges	12	12	0
Scoreboard	9	9	0
Total	49	49	0

Before this final run, each module was also run on its own, and the groups were run through the runner both one test at a time and in parallel.

9. Problems found during development

Every difference between what I expected and what happened turned out to be an incorrect assumption in my own tests. I did not find a defect in CTFd in the areas I tested. The main problems and their fixes were:

Problem	Cause	Fix
Wrong-flag message changed between runs	After a user solves a challenge, the message becomes "Incorrect but you already solved this"	Each test registers a new user first
A solve count check failed	The count includes every user who solved the challenge, not just the current one	Removed the count checks
Empty flag test failed	CTFd shows "Incorrect" for an empty flag, not nothing	Updated the expected result and the notes
Blank fields and bad email formats showed no server message	The browser stops these forms before they are sent	Expected results changed to "form is not submitted"

Problem	Cause	Fix
Scoreboard tests passed without checking anything	The Place number is in a header cell, so the table was read as empty	Read both header and data cells, and fail if no rows are found
Column header tests failed in parallel runs	The header text was read before the page finished drawing	Wait until the header text is present
Profile link click failed	The user was at the bottom of a long table and the link was covered	Scroll to the link before clicking

The scoreboard problem was the most useful lesson. Tests that pass on an empty result can look correct while checking nothing, so the tests now assert that data was actually read.

10. Limitations and future work

- The tests work through the browser, so they are functional tests of the interface rather than isolated tests of individual Python functions.
- Two error guessing cases in the Authentication notes (a script tag and a SQL-style string as the username) are designed but not yet automated.
- The username upper length limit was not tested, because the code does not enforce one.
- All users in the scoreboard data have 100 points, so the ordering check only proves that scores never increase down the table. A second challenge worth a different number of points would make it stronger.
- CTFd uses SQLite, so parallel runs are best kept to two or three workers.
- The tests assume the admin account and Test Challenge described in the setup section.

11. Repository contents

- `tests/` : the Selenium test files.
- `data/` : the CSV files that the tests read.
- `notes/` : the test design notes for each module.

- `docs/`: the setup videos and the Excel test case sheet.
- `run_tests.py` and `test_config.py`: the runner and its configuration.
- `requirements.txt` and `README.md`: dependencies and run instructions.